

Documento de Testing

Objetivo

Este documento establece cómo se ejecutan y mantienen los tests del proyecto. Además, describe el enfoque de pruebas unitarias y de la capa MVC en el backend.

Cómo Ejecutar los Tests

El proyecto utiliza Maven para la gestión de las pruebas. Las siguientes instrucciones ejecutan únicamente los tests unitarios y de `@WebMvcTest` del backend. **El frontend no tiene pruebas en este repositorio.**

Sistema Operativo	Comando de Ejecución	Descripción
Windows	<code>./mvnw.cmd -DskipITs -DfailIfNoTests=false test</code>	Ejecuta tests unitarios y de MVC del backend.
Linux/macOS	<code>./mvnw -DskipITs -DfailIfNoTests=false test</code>	Ejecuta tests unitarios y de MVC del backend.

Comandos Útiles

Comando	Descripción
<code>./mvnw.cmd -Dtest=CartServiceImplTest test</code>	Ejecuta únicamente el test <code>CartServiceImplTest</code> .
<code>./mvnw.cmd -X test</code>	Muestra logs detallados de la ejecución de los tests.

Frameworks y Herramientas

Se utilizan las siguientes herramientas para el desarrollo de pruebas:

- **JUnit 5:** Para la escritura de pruebas unitarias y de integración.
- **Mockito:** Para la creación de mocks de dependencias.

- **Spring Boot Test:** Para la gestión del contexto de pruebas, especialmente para `@WebMvcTest` y `MockMvc`.

Tipos de Pruebas

Los tests en el backend se clasifican según la capa:

Capa	Tipo de Test	Foco	Convención de Nombres
Servicios	Unitarias	Validación de lógica de negocio, excepciones y flujos.	<code>service.impl/*Test</code>
Controladores	Pruebas MVC	Validación de <i>endpoints</i> HTTP utilizando <code>MockMvc</code> . Los filtros de seguridad reales están deshabilitados.	<code>controller/*WebMvcTest</code>
Seguridad JWT	Unitarias	Pruebas de generación y validación de tokens.	<code>security/JwtTokenProviderTest</code>

Patrones Clave de Testing

A continuación, se describen los patrones esenciales para el desarrollo de tests en el proyecto:

- **Deshabilitar filtros en `@WebMvcTest`:** Para aislar la prueba del controlador de la cadena de filtros de seguridad, se debe usar la anotación: `@AutoConfigureMockMvc(addFilters = false)`.
- **Mockear dependencias de seguridad:** Cuando el contexto de prueba de MVC requiere la presencia de beans de seguridad (como `ProveedorTokenJwt` y `ServicioDetallesUsuario`), estos deben ser mockeados con `@MockBean`.
 - *Ejemplo:* Ver líneas 28-39 en `<span type="placeholder" placeholder-type="file"></span>`.
- **Uso de `@WithMockUser`:** Esta anotación se utiliza para simular un usuario autenticado cuando se evalúan reglas de método basadas en roles o permisos (por ejemplo, `@PreAuthorize("hasRole('ADMIN')")`).

Cobertura Actual (Resumen)

La cobertura de pruebas del backend incluye:

Controladores

- Autenticación
- Carrito
- Productos
- Categorías
- Regiones
- Comunas
- Proveedores

Servicios

- Productos
- Categorías
- Regiones
- Comunas
- Carrito

Seguridad

- Proveedor de tokens JWT

Buenas Prácticas

1. **Aislamiento en @WebMvcTest:** Mantener las pruebas de controlador aisladas del *stack* completo de Spring. Importar o *mockear* solo las dependencias estrictamente necesarias.
2. **Validación de Excepciones:** Validar explícitamente los mensajes y tipos de excepciones esperadas en los servicios (ej. `RecursoNoEncontradoException`, `PeticionInvalidaException`).
3. **Evitar Datos Reales:** Utilizar *builders* y `Mockito.when` para simular dependencias y datos, evitando cualquier dependencia a datos reales de la base de datos.

Solución de Problemas Comunes

Problema	Solución Recomendada
Error de <code>ApplicationContext</code> por beans de seguridad ausentes.	Añadir <code>@MockBean</code> de <code>ProveedorTokenJwt</code> y <code>ServicioDetallesUsuario</code> en la clase de test.

Problema	Solución Recomendada
Roles inconsistentes al simular usuarios.	Usar el rol correcto en cada test, acorde a la regla de método: <code>@WithMockUser(roles = {"CLIENTE"})</code> .
Las pruebas de subida de imágenes no escriben archivos binarios.	El flujo de prueba valida el manejo y la respuesta HTTP, no la escritura física en el directorio <code>uploads/images</code> .

Estado Actual

Todos los tests del backend se ejecutan exitosamente.

Tests run: 92, Failures: 0, Errors: 0.